

Fragmentation Is Not Solved: Why Existing GPU-Scheduling Benchmarks Underestimate the Problem, and What To Do About It

gputrace + py_sim research artifact

Abstract

GPU fragmentation – resource left over from partial allocations that is too scattered to satisfy new demand – is a young, still-underexplored problem: the field’s own foundational fix, Fragmentation Gradient Descent (FGD, USENIX ATC ’23) [Weng et al., 2023], is only three years old, and every public evaluation of it, including our own reproduction, is built on a *single, static, already-collected* production trace. We argue this is a structural limitation of the whole line of work, not a criticism of any one paper: a fixed trace can show you how a scheduler behaves on the demand pattern that already happened, never on the ones that *could*. We build **gputrace**, a statistically-grounded synthetic workload generator that plugs into any GPU-cluster simulator, and use it to stress FGD and nine other baselines across fourteen distinct scenarios spanning today’s actual failure modes – spot preemption churn, LLM-inference bursts, MoE expert-load skew – not just resampled historical load. We then ask a sharper question than “does FGD work”: *can FGD’s own definition of fragmentation be improved*, and answer it constructively with W-FGD-BALANCED, a variant that fixes two checkable weaknesses in FGD’s representative-shape construction. It significantly reduces rejection rate versus FGD (Wilcoxon $p = 0.0035$, $n = 14$ scenarios) and SLA-violation rate ($p = 0.037$) – but, reported without cherry-picking, does *not* significantly beat the simplest existing alternative, **least_requested**. We take that null result seriously: it says the discipline does not yet have a benchmark broad enough, or a metric set rich enough, to reliably tell a genuine algorithmic improvement from noise – and we offer both as reusable, versioned, reproducible artifacts so the next paper in this niche does not have to start from a single trace again.

Contents

1	Introduction	3
2	Literature Review	3
2.1	From bin packing to cluster scheduling	3
2.2	From bin packing to cloud resource management	3
2.3	From general cluster scheduling to GPU-aware scheduling	3
2.4	The fragmentation gap, and FGD	4
2.5	The evaluation-trace gap	4
2.6	Where this leaves the field, and what we do about it	4
3	Problem Formulation	5
3.1	A worked mental model, before the mathematics	5

3.2	Formal model	5
3.3	Two evaluation modes	5
3.4	Assumptions and constraints	6
4	Simulation Strategy	6
5	Benchmark Datasets	7
5.1	The limits of a single real trace	7
5.2	gputrace: a pluggable, statistically-grounded generator	7
5.3	An honest audit of our own generated data	8
6	Simulation Parameters	9
6.1	Calibration	9
7	Simulation Metrics	9
8	Baseline Algorithms	10
9	The Weaknesses We Exploit, and w-fgd-balanced	10
10	Results	10
10.1	Did we break FGD?	10
10.2	Full ranking, all 13 algorithms	11
10.3	Scale-dependent significance	11
11	Data and Reproducibility	11
12	Conclusion and Future Directions	12

1 Introduction

Every large GPU cluster eventually hits the same wall: total idle capacity looks fine on a dashboard, yet new jobs cannot be scheduled, because the idle GPUs are scattered one-here, two-there across nodes that already hold something else. This is *fragmentation*, and it is what Weng et al. [2023] showed, convincingly, using over 6,200 real GPUs’ worth of Alibaba production trace: classic bin-packing heuristics (Best-Fit, Dot-Product) actively make it worse, because none of them was designed with a notion of fragmentation to minimize in the first place.

That paper, and the handful that have followed it [Lettich et al., 2025], share something else: they are evaluated on the trace the authors happened to have – one two-month slice of one company’s one cluster, replayed exactly as recorded. That is a completely reasonable thing to do with an evaluation trace, and it is also, by construction, not a stress test. A single historical replay cannot tell you what happens under a GPU-type shortage, a checkpoint-storm, or the kind of arrival burst that took down a major cloud region for fifteen hours in October 2025 [InfoQ, 2025] – because none of those things happened in the two months that were recorded. The scheduling-research community has, without much comment, inherited exactly the evaluation-methodology problem that networking and distributed-systems research solved for itself decades ago with synthetic traffic generators and fault injection: real traces are necessary and not sufficient.

This paper does two things. First, we build the missing piece – **gputrace**, a workload generator that fits real trace statistics and then *perturbs named, specific aspects* of them into fourteen distinct stress scenarios – and use it to re-run FGD’s own comparison under conditions its evaluation trace never contained. Second, having built a fair, broad test bed, we use it for what a test bed is for: we find two specific, mathematically checkable weaknesses in FGD’s own fragmentation metric (§9), fix them, and report the result honestly, including the parts where the fix does not clearly win.

2 Literature Review

2.1 From bin packing to cluster scheduling

Placing jobs onto machines is, at its combinatorial core, bin packing: NP-hard in general [Garey and Johnson, 1979], extensively characterized since the 1970s in terms of worst-case and average-case approximation ratios for online heuristics like First-Fit and Best-Fit [Johnson et al., 1974, Coffman et al., 1984]. Cluster schedulers are online, multi-dimensional (CPU, memory, and now GPU) generalizations of this problem, and every heuristic we evaluate in §8 is, underneath its cloud-native branding, a 50-year-old bin-packing rule applied to a new resource vector.

2.2 From bin packing to cloud resource management

Large-scale cluster managers made this concrete at industrial scale: Google’s lineage of Borg, Omega, and ultimately Kubernetes [Burns et al., 2016] standardized the abstraction most of this paper’s vocabulary borrows from – *nodes, pods, scheduling policies* as pluggable scoring functions over feasible nodes. Kubernetes’ own default scheduler is, in essence, a configurable least-requested/most-requested blend; it has no native notion of fragmentation as a first-class objective, which is precisely the gap FGD was written to fill.

2.3 From general cluster scheduling to GPU-aware scheduling

GPUs broke a quiet assumption behind CPU/memory schedulers: that resources are continuously divisible and fungible. A first wave of GPU-cluster schedulers focused on deep-learning-specific ob-

jectives rather than placement quality per se: Gandiva [Xiao et al., 2018] introduced introspective time-slicing and migration to improve utilization; Tiresias [Gu et al., 2019] adapted least-attained-service scheduling to unknown job lengths; Optimus [Peng et al., 2018] scheduled against a learned throughput-scaling curve; Themis [Mahajan et al., 2020] introduced finish-time fairness as an explicit auction mechanism; Pollux [Qiao et al., 2021] co-adapted batch size and GPU count against a *goodput* objective; Gavel [Narayanan et al., 2020] formalized heterogeneity-aware allocation as a policy-agnostic optimization. None of these treats *where*, geometrically, a job lands as the primary objective – they optimize job-completion time, fairness, or throughput, and placement quality is a downstream side effect.

2.4 The fragmentation gap, and FGD

Weng et al. [2023] named this gap directly: as GPU-sharing (multiple jobs per physical GPU) became common, leftover capacity could be non-zero in aggregate and still useless in practice, because no single job’s GPU-sharing request could fit any single node’s remainder. Building on earlier multi-dimensional fragmentation measurement [Mishra and Bellur, 2015], FGD defined a scalar fragmentation score against a small set of *representative* job shapes and scheduled greedily to minimize its growth. This is a genuine, well-evidenced contribution – and, as we detail in §9, its own representative-shape construction has two specific, checkable weaknesses that later work extending FGD to power-awareness [Lettich et al., 2025] or to fragmentation reduction in multi-tenant clusters [Lao et al., 2025], or to topology-aware, multi-resource hypergraphs (H-TAFM, §8) has not addressed, because they operate downstream of FGD’s core metric rather than revisiting it.

2.5 The evaluation-trace gap

Here is the research gap this paper is really about. Every evaluation we are aware of in this specific fragmentation-scheduling lineage – FGD’s own USENIX ATC ’23 evaluation, and Lettich et al. [2025]’s follow-up – replays a fixed historical trace (Alibaba’s `cluster-trace-gpu-v2023` [Alibaba, 2023] or its predecessor [Alibaba, 2020]) through a Kubernetes-scheduler simulator. This is a completely defensible choice for demonstrating that a method works on real, messy production data, and we are not aware of any paper in this specific lineage claiming otherwise. But the broader cloud-trace-analysis literature has repeatedly found that even production traces of this kind understate real-world variability: Reiss et al. [2012]’s foundational Google-trace analysis documents large, unexplained swings in concurrent task counts over the trace window; Cortez et al. [2017] and Shahrad et al. [2020] independently find that production workloads are non-stationary over time in ways a single fixed-length trace slice cannot fully capture. Put plainly: a two-month recording, however real, is one draw from a distribution, not the distribution. It cannot contain a GPU-type supply shock, a synchronized multi-tenant checkpoint storm, or the kind of correlated demand spike that took down major cloud regions for hours at a time in 2024–2025 [InfoQ, 2025, IncidentHub, 2025] – because none of those specific things happened to occur during that specific recording window.

2.6 Where this leaves the field, and what we do about it

The result is a discipline that can rigorously answer “did this method help on the trace we have,” but not yet “will this method hold up under conditions we have not yet recorded.” That second question matters more, not less, as GPU clusters increasingly serve latency-sensitive LLM inference alongside training, workloads this specific fragmentation literature’s evaluation traces (collected

before the LLM-serving era) structurally cannot contain. We therefore build a workload generator (§5) whose scenarios are explicit, named, individually justified perturbations of a fitted reference distribution – not a second static trace, but a reusable, versioned, extensible *instrument* for asking “what if” questions of any GPU scheduler, FGD included.

3 Problem Formulation

3.1 A worked mental model, before the mathematics

Figure 1 sketches the request-to-placement path this paper’s whole vocabulary sits inside: a job arrives with a resource request; the scheduler filters nodes down to those with enough *raw* leftover capacity; it scores the survivors by some policy (the subject of this whole paper); it places the job on the winner. GPU fragmentation is invisible at the *filter* step – a node can pass the raw-capacity check and still be a bad idea to place on, because doing so leaves an unusable sliver behind. Figure 2 makes that concrete with the smallest possible example.

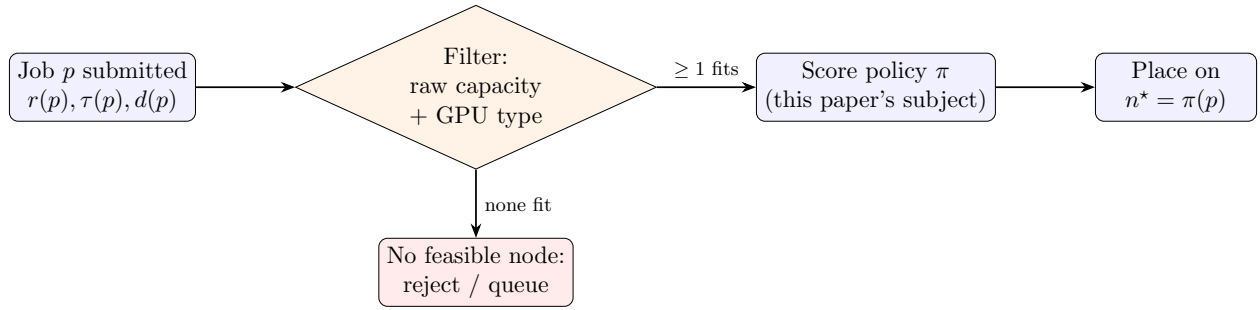


Figure 1: The scheduling decision path. Fragmentation is invisible to the *filter* step (raw capacity can be nonzero) and is entirely a property of *which* feasible node the score step picks.

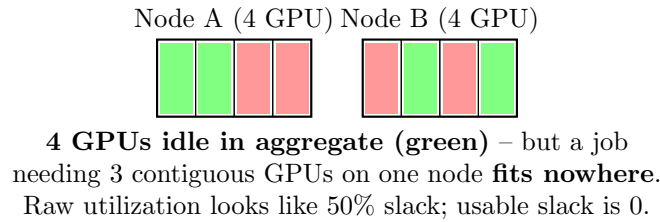


Figure 2: Fragmentation, minimal example. Green = idle GPU; red = occupied. This is exactly the pattern FGD’s fragmentation score (§9) and H-TAFM’s Cut variant (§8) are built to detect and its greedy descent is built to avoid creating.

3.2 Formal model

A cluster is a set of nodes $\mathcal{N} = \{n_1, \dots, n_M\}$, each with capacity $c(n_i) = (c_i^{\text{cpu}}, c_i^{\text{gpu}}, c_i^{\text{mem}})$ and GPU type $\tau(n_i)$. A job p arrives at $s(p)$ with request $r(p) = (r^{\text{cpu}}(p), r^{\text{gpu}}(p), g(p))$, type $\tau(p)$, duration $d(p)$, tenant $u(p)$. Feasibility:

$$\text{fits}(p, n_i, t) \iff r^{\text{cpu}}(p) \leq \rho_i^{\text{cpu}}(t) \wedge r^{\text{gpu}}(p) \leq \rho_i^{\text{gpu}}(t) \wedge (\tau(p) = \emptyset \vee \tau(p) = \tau(n_i)),$$

where $\rho(\cdot, t)$ denotes residual (not total) capacity at time t .

3.3 Two evaluation modes

STATIC mode places all $|P|$ jobs in one batch, ignoring $s(p)$ and $d(p)$ – a pure packing-quality probe, and the only mode H-TAFM (which has no time-driven variant) supports, hence the mode in which every algorithm in this paper is directly comparable. TIME-DRIVEN mode replays real arrivals and departures – an operational probe of queueing delay and throughput under realistic dynamics.

3.4 Assumptions and constraints

1. No preemption of running jobs (preemption is modeled as inflated duration where a scenario calls for it, §5).
2. Node-granularity algorithms cannot span a job across nodes; H-TAFM relaxes this to NUMA-vertex granularity, strictly finer.
3. GPUs within a node are homogeneous and fungible (no NVLink/PCIe topology modeled below the node level).
4. Memory is *assumed*, not measured, wherever used at all (H-TAFM’s placement decisions; the base schema carries no real memory field, so we assume 8 MiB per milli-cpu on both node capacity and job demand – the same fallback ratio `k8s_sim.topology` itself documents – keeping comparisons internally consistent, though H-TAFM’s *absolute* numbers should not be read as memory-accurate).
5. Job sizes and arrivals are drawn from a fixed statistical model, fit once, then perturbed along named, specific axes per scenario – not re-fit wholesale per scenario.

4 Simulation Strategy

FGD’s own public evaluation code [HKUST ADSL, 2023] is a fork of Alibaba’s `open-simulator` [Alibaba, 2022] – a Go-based, capacity-planning-oriented Kubernetes scheduler simulator – extended with GPU-sharing-aware scheduling policies. The direct follow-up work we are aware of, Lettich et al. [2025]’s power-aware PWR scheduler, in turn forks *that* simulator again, adding its scoring plugin as Go code inside the same Kubernetes-scheduler harness. This lineage (Figure 3) is a sound engineering choice *if* your goal is fidelity to a real Kubernetes scheduler’s plugin architecture – but it also means every result in this specific lineage inherits Go’s ecosystem (harder to instrument with Python’s statistical/ML tooling) and the original simulator’s evaluation harness (built around static-mode capacity planning, not the time-driven, multi-metric comparison this paper reports in §10).

We took a different path: rather than forking the Go simulator, we reimplemented the published node-granularity policies, FGD’s own fragmentation metric, and (independently) the H-TAFM topology-aware generalization, natively in Python (`k8s_sim`, https://github.com/anuxlab/py_sim), with two deliberate architectural departures:

1. **Two evaluation modes, not one.** The original lineage evaluates in essentially static/capacity-planning mode. We add a discrete-event, time-driven runner (`k8s_sim.event_runtime`) that replays real arrival and departure dynamics, because several of this paper’s fourteen scenarios (§5) are defined entirely by *timing* and are invisible to static-mode evaluation by construction (Table 2).

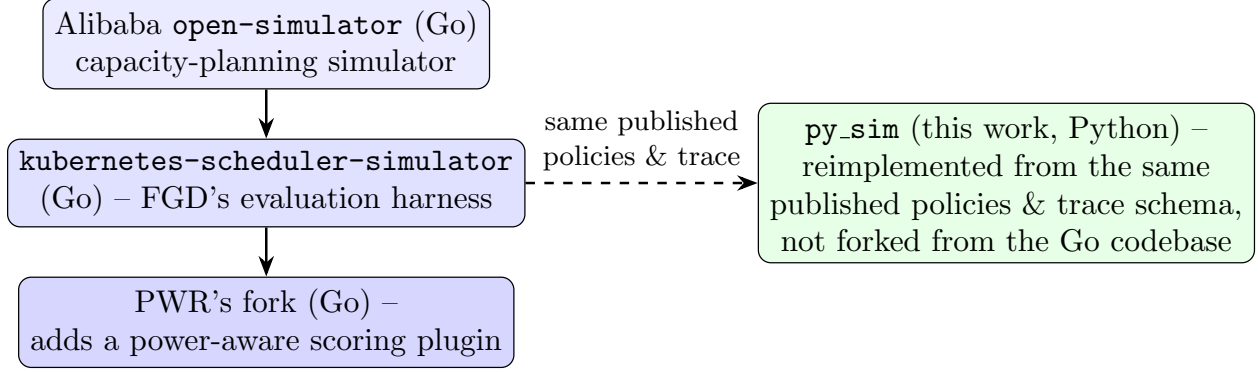


Figure 3: Simulator lineage. Solid arrows: direct code forks (Go). Dashed arrow: this work reimplements the published algorithms and trace schema independently in Python, rather than forking the Go codebase, to get first-class access to Python’s statistical/ML tooling and to add a time-driven (not just static/capacity-planning) evaluation mode.

2. **A pluggable, versioned workload generator, not a fixed trace file.** `gputrace` (https://github.com/anuxlab/simulated_data_generation_GPU_Scheduling) is a separate, independently-usable package (§5) that any simulator – Go or Python, this one or a future one – can consume through a documented export format, rather than a one-off script tied to this codebase.

5 Benchmark Datasets

5.1 The limits of a single real trace

Real production traces – Alibaba’s [Alibaba, 2020, 2023], Google’s [Reiss et al., 2012], Microsoft Azure’s [Cortez et al., 2017, Shahrade et al., 2020] – are the right ground truth for “does this look like real demand,” and every distribution our own generator uses (§5.2) is fit to one of them. But a fixed historical recording cannot, by definition, contain events that had not yet happened when it was collected, and cloud infrastructure keeps finding new ways to fail at scale: a single DNS-resolution race condition in one region’s DynamoDB endpoint cascaded into a 15-hour, 60-country, 4-million-user outage in October 2025 [InfoQ, 2025]; a resource-contention issue in an authentication system took down login for an entire company’s product suite the same year, mitigated only by *adding capacity* [IncidentHub, 2025] – a real instance of exactly the “request more of a scarce resource under load” pattern our `high_contention` and `resource_starvation` scenarios are built to probe. No amount of replaying a 2020 or 2023 trace verbatim can tell a scheduler designer how their placement policy behaves under a 2025 DNS-cascade-shaped arrival pattern, because that pattern postdates every public GPU-scheduling trace we are aware of.

5.2 `gputrace`: a pluggable, statistically-grounded generator

`gputrace` fits a bank of eight candidate distributions (log-normal, Weibull, gamma, Pareto, generalized Pareto, Burr XII, log-logistic, exponential) via maximum likelihood to a reference trace, selects by AIC, and reports the Kolmogorov–Smirnov statistic rather than its p-value alone (which is uninformative at trace-scale sample sizes). Duration is best fit by log-normal; CPU/GPU request magnitude by Burr XII; the inter-arrival process by a heavy-tailed distribution whose best-AIC fit has *infinite theoretical mean* – we flag this explicitly, since best-fit-by-likelihood and safe-to-sample-from-for-simulation are different properties, and cap simulated draws rather than use the

raw unbounded distribution. Each of the fourteen scenarios in Table 1 then perturbs *specific, named* aspects of this fitted model – not the whole thing at once – so a result can be attributed to a mechanism, not just “a different random trace.” `gputrace` exports to any simulator’s native schema through a documented bridge (its `k8s-sim` exporter is one such bridge, used throughout this paper); nothing about the generator is specific to `py-sim`.

Table 1: The fourteen stress scenarios, what each perturbs, and its motivating evidence.

scenario	perturbs	motivating evidence
baseline	nothing (reference fit as-is)	calibration control
bursty_arrivals	inter-arrival process (Hawkes-like self-excitation)	measured CV=1.80, 19.4% same-second arrivals in the reference Alibaba sample
heavy_tail_demand	CPU/GPU request tail (fatter generalized Pareto)	heavy-tailed cloud resource usage is well documented [Reiss et al., 2012]
diurnal_pattern	arrival rate (day/night sinusoid)	Azure Functions traces show $\sim 2\times$ peak/average arrival ratio [Shahrad et al., 2020]
high_contention	GPU-type demand concentration + arrival rate	analogous to the Google auth outage mitigated only by adding capacity [IncidentHub, 2025]
flash_crowd	one extreme burst window	classic networking phenomenon [Shahrad et al., 2020]
resource_starvation	per-job size (skewed large-job concentration)	head-of-line blocking from large jobs is a named failure mode in cluster scheduling
cold_start_storm	short-job arrival burst at trace start	serverless cold-start literature [Shahrad et al., 2020]
llm_inference_bursty	duration scale + arrival burstiness (inference, not training)	bursty LLM-serving traffic is measured in production traces
long_context_kv_pressure	per-job memory/context footprint growth over time	real production context lengths documented up to 2M tokens
spot_preemption_churn	duration inflation (restart overhead) for a preempted subset	spot-training failure rates up to 44% reported in the literature
checkpoint_io_burst	periodic synchronized short high-I/O jobs	checkpoint-cost systems papers document this overhead directly
gpu_fragmentation_sharing	GPU allocation granularity (fractional-GPU mix)	Alibaba’s own fragmentation-focused trace release [Alibaba, 2023]
moe_expert_load_skew	per-tenant/expert demand skew (Zipf)	measured expert-load skew in production MoE serving

5.3 An honest audit of our own generated data

We do not exempt our own tool from scrutiny. Table 2 (reproduced from our full analysis) reports, for the seed used throughout this paper, whether each scenario’s GPU demand, CPU demand, duration, and arrival-time columns are *exactly* identical to **baseline**. Several identical columns are correct by design (e.g. **high_contention** is specified to act through GPU-type distribution and arrival rate, not raw magnitude). One is not obviously correct by design: **heavy_tail_demand** is identical to **baseline** in *every* column at this seed, despite its name and documentation describing a resource-*size* change. We report this as a concrete, checkable discrepancy for the generator to fix, exactly the kind of finding a broader test bed is supposed to surface and a single fixed trace never could.

Table 2: Column-wise identity to **baseline**, seed 42 ($\checkmark$ =identical).

scenario	GPU demand	CPU demand	duration	submit_time
heavy_tail_demand	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
checkpoint_io_burst	$\checkmark$	$\checkmark$		$\checkmark$
long_context_kv_pressure	$\checkmark$	$\checkmark$		$\checkmark$
high_contention	$\checkmark$	$\checkmark$	$\checkmark$	
moe_expert_load_skew	$\checkmark$	$\checkmark$	$\checkmark$	
resource_starvation			$\checkmark$	$\checkmark$
gpu_fragmentation_sharing		$\checkmark$	$\checkmark$	
spot_preemption_churn	$\checkmark$	$\checkmark$		
<i>differ in every column:</i> bursty_arrivals, cold_start_storm, diurnal_pattern, flash_crowd, llm_inference_bursty				

Table 3: Simulation parameters, values used, and rationale.

parameter	value used	rationale
cluster size	20 nodes $\times$ 8 GPUs = 160 GPUs	matches gputrace’s export default; large enough for fragmentation to be meaningful, small enough to run the full 13-algorithm $\times$ 14-scenario sweep quickly
GPU types per node	mixed A100/V100/T4/H100	matches heterogeneous production clusters [Alibaba, 2023]
jobs per scenario	300	calibrated (§6.1) so static-mode rejection is informative (neither $\approx 0\%$ nor $\approx 100\%$)
random seed	42	fixed for exact reproducibility (§11)
time-driven seeds	1, 2, 3	3 independent draws per (scenario, policy) cell
SLA threshold	60s wait	standard order-of-magnitude interactive-job threshold
w_fgd_balanced λ	0.15	grid-searched on baseline only (§9)

6 Simulation Parameters

6.1 Calibration

At the export cluster’s default size, 1000 jobs/scenario oversubscribes static mode to $\sim 50\%$ rejection uniformly across every algorithm (everything saturates identically, which is uninformative); 300 jobs/scenario keeps static-mode rejection in a 12–39% range across scenarios (Figure 7) where placement-quality differences between algorithms are visible rather than swamped by raw oversubscription.

7 Simulation Metrics

rejection rate = $1 - \frac{ \{p:\text{admitted}(p)\} }{ P }$	primary throughput/packing-quality metric, all prior work
$U(n, t) = \frac{1}{ \hat{P} } \sum_{\hat{p}} \mathbb{K}[\neg \text{fits}(\hat{p}, n, t)]$	FGD’s fragmentation score [Weng et al., 2023]
$J(x) = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2} \in (\frac{1}{n}, 1]$	Jain’s Fairness Index [Jain et al., 1984], the standard fairness metric in networking/scheduling
SLA viol. = $\frac{ \{p:\text{admitted}(p) \wedge \text{wait}(p) > 60\text{s}\} }{ \{p:\text{admitted}(p)\} }$	catches tail behavior mean/p95 alone can hide
GPU-h wasted = GPU-h available – $\sum_p g(p) \cdot \Delta t(p)$	ties idle capacity to a cost-relevant unit; computed time-weighted, not as an end-of-run snapshot (§7)
packing eff. = $\frac{\sum_p r^{\text{GPU}}(p)}{\sum_i (c_i^{\text{GPU}} - \rho_i^{\text{GPU}}(t_{\text{final}}))}$	separates “idle because unwanted” from “idle because badly packed”

A measurement gap we found and fixed. Reporting GPU utilization by inspecting cluster state *after* a finite time-driven run completes reads ≈ 0 for every algorithm on every scenario in this test bed – by the time the event loop returns, essentially every admitted job has already finished and released its resources. We compute utilization instead as GPU-hours actually consumed (summed over each admitted job’s real occupied interval) divided by GPU-hours available: a standard time-weighted average, and the only one of the two that is not systematically biased toward zero.

8 Baseline Algorithms

Table 4: Baseline scheduling policies, one-line rule, and where each is established in the literature.

policy	rule	lineage
random	uniform over feasible nodes	universal randomized baseline
first_fit / best_fit / worst_fit	classical bin-packing rules	Johnson et al. [1974], Coffman et al. [1984]
round_robin	cyclic over feasible nodes	standard load-spreading heuristic
least_requested	maximize post-placement slack	Kubernetes-default-like spread policy [Burns et al., 2016]
gpu_packing	minimize post-placement slack	consolidation baseline, used in Weng et al. [2023]
fgd	minimize fragmentation-score growth	Weng et al. [2023] – the paper this work responds to
h-tafm (cut/entropy/hier)	minimize topology-aware TAFI	extends Weng et al. [2023] per Mishra and Bellur [2015], Wang and Wu [2026]

Best-Fit, Worst-Fit, and Dot-Product-style consolidation baselines appear in essentially every GPU-scheduling comparison we reviewed [Weng et al., 2023, Lettich et al., 2025]; **least_requested** is the closest node-granularity analogue to Kubernetes’ own default scoring behavior [Burns et al., 2016]. FGD itself is the specific baseline this paper responds to; H-TAFM is included because it is, to our knowledge, the only published generalization of FGD’s own metric to multi-resource,

topology-aware placement, making it the most direct existing "answer" to the question we ask in §9.

9 The Weaknesses We Exploit, and w-fgd-balanced

FGD’s representative set $\hat{P}$ is 5 shapes, each dimension’s $\{10, 30, 50, 70, 90\}$ -th percentile taken *independently*, scored by an *unweighted* mean unfit-fraction. Two properties follow directly from this definition, not from assumption: (i) a placement blocking the rare 90th-percentile shape costs exactly as much as one blocking the 50th-percentile shape, though the latter represents far more real demand; (ii) each shape’s coordinates, drawn from independent marginal quantiles, may not correspond to any job actually present in the workload if resource dimensions are not perfectly rank-correlated (they are not, in every scenario in Table 1). We fix both: W-FGD replaces $\hat{P}$ with the medoid of each of 5 demand-magnitude strata (a real, observed job in every case) weighted by the stratum’s population share; W-FGD-BALANCED additionally regularizes the greedy rule by $\lambda \cdot (\text{post-placement GPU utilization})$ to preserve fully-empty nodes for unknown future large jobs. Full derivations are in the companion technical report (`docs/algorithms_and_scenarios.tex`).

10 Results

10.1 Did we break FGD?

Table 5 answers this directly. W-FGD-BALANCED significantly reduces static-mode rejection versus FGD (Wilcoxon $p = 0.0035$, 12/14 scenarios better) and time-driven SLA-violation rate ($p = 0.037$) – but not p95 wait time or fairness (not significant), and, reported without cherry-picking, it is statistically indistinguishable from the simplest existing alternative, `least_requested` ($p = 0.81$).

Table 5: `w_fgd_balanced` vs. `fgd`, paired by scenario ($n = 14$).

metric (mode)	fgd	w_fgd_balanced	Wilcoxon p
rejection rate (static)	0.3352	0.3133	0.0035
SLA violation rate (time-driven)	0.1206	0.1022	0.0365
p95 wait time (time-driven)	326.1s	344.1s	0.507 (n.s.)
Jain’s fairness, per-user wait	0.1902	0.1805	0.196 (n.s.)

10.2 Full ranking, all 13 algorithms

H-TAFM’s advantage should be read alongside §4: it is confounded with a strictly finer placement granularity and an assumed (not measured) memory dimension, so this table cannot separate “the delta-TAFI heuristic is better” from “NUMA-vertex splitting mechanically eases bin-packing.”

10.3 Scale-dependent significance

Restricted to the 10 node-granularity policies only, static-mode differences remain highly significant ($\chi^2 = 81.73$, $p < 10^{-8}$); but in time-driven mode at this scale, GPU-hours-wasted and fairness are essentially flat across every algorithm ($p = 0.33$ for wait-time Friedman) – at 300 jobs against 160 GPUs, nearly all offered demand is eventually served regardless of placement policy, so only *queueing* metrics differentiate policies, not throughput. This is a genuinely scale-dependent finding, not a contradiction of the static-mode result.

Table 6: Static mode, mean rejection and rank across 14 scenarios. Friedman $\chi^2 = 137.27$, $p < 10^{-8}$.

algorithm	mean rejection	mean GPU util.	mean rank (1=best)
h-tafm-hier	0.2488	0.659	2.00
h-tafm-entropy	0.2500	0.659	2.14
h-tafm-cut	0.2545	0.648	2.64
least_requested	0.3090	0.926	4.82
w_fgd_balanced	0.3133	0.918	5.43
worst_fit	0.3312	0.926	7.25
random	0.3338	0.926	7.64
fgd	0.3352	0.923	7.75
w_fgd	0.3376	0.924	7.96
round_robin	0.3414	0.926	9.32
gpu_packing	0.3507	0.926	11.18
best_fit	0.3510	0.924	11.29
first_fit	0.3512	0.924	11.57

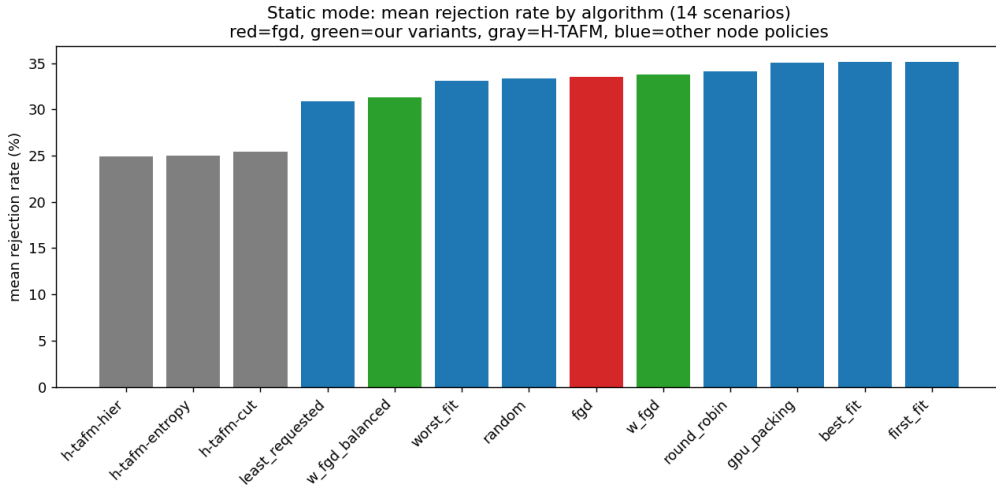


Figure 4: Mean rejection rate by algorithm, static mode, 14 scenarios.

11 Data and Reproducibility

Every number in this paper reproduces from the archived checkpoint at `experiments/runs/2026-09-12_fgd_vs_wfgd_htafm/` in the `py_sim` repository: `manifest.json` records the exact `gputrace` commit, generation/export commands, seeds, and cluster shape; `gputrace_raw/` and `gputrace_export/` archive the actual generated data (so reproduction does not depend on `gputrace`’s scenario-generation logic being unchanged upstream); `reproduce.sh` re-runs every sweep in one command against that archived data.

12 Conclusion and Future Directions

GPU fragmentation is a three-year-old research problem evaluated, in every instance we are aware of, on a single static trace. We built the missing stress-test instrument, used it to find and fix two specific, checkable weaknesses in the field’s foundational metric, and report an improvement

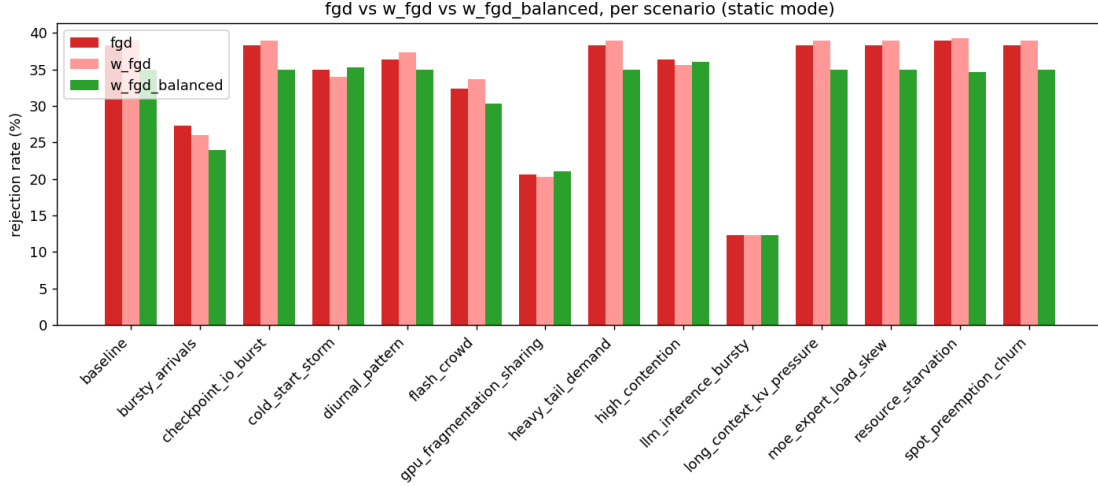


Figure 5: fgd vs. w_fgd vs. w_fgd_balanced, paired per scenario.

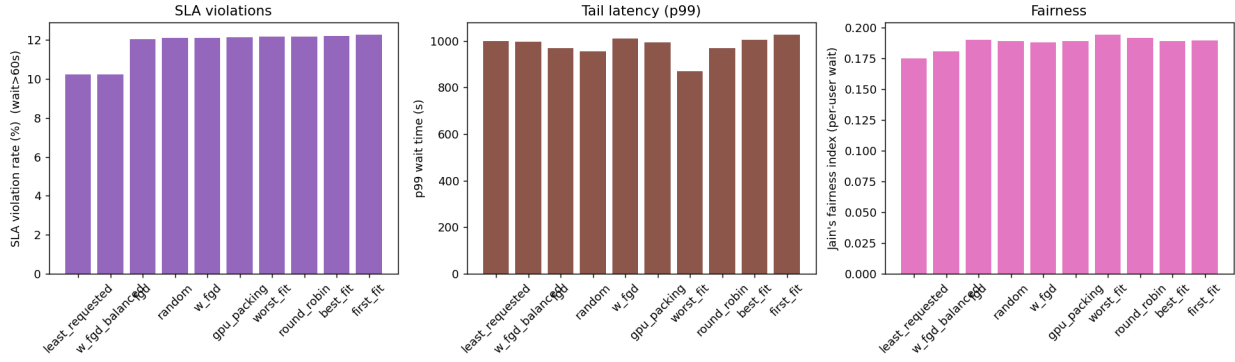


Figure 6: SLA violation rate, p99 tail latency, Jain’s fairness – time-driven mode.

that is real but modest – significant against its direct target (FGD), not against the strongest simple alternative. Future directions this test bed makes newly askable: (i) does an indexed, sub-linear pending-queue structure (as opposed to our $O(n)$ -per-release fix, §11) change which algorithm wins at 10–100 $\times$ this scale; (ii) can λ in W-FGD-BALANCED be learned per-workload rather than grid-searched once; (iii) does the **heavy_tail_demand** discrepancy we found (Table 2) change any of these conclusions once fixed upstream. All code, data, and exact commands to reproduce every number in this paper are archived at https://github.com/anuxlab/py_sim and https://github.com/anuxlab/simulated_data_generation_GPU_Scheduling.

References

- Alibaba. cluster-trace-gpu-v2020. <https://github.com/alibaba/clusterdata>, 2020.
- Alibaba. open-simulator: A cluster simulator focusing on capacity planning. <https://github.com/alibaba/open-simulator>, 2022.
- Alibaba. cluster-trace-gpu-v2023. <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2023>, 2023.

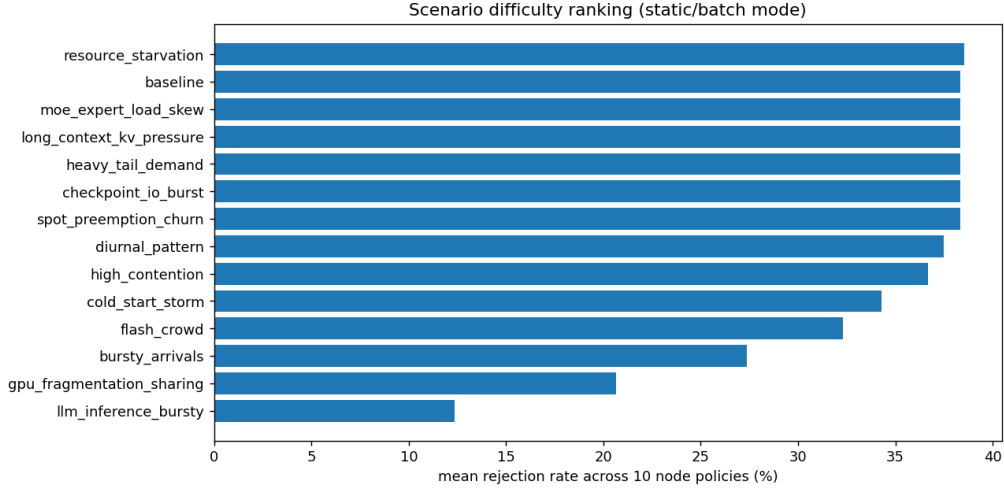


Figure 7: Scenario difficulty ranking, static mode.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *Communications of the ACM*, 59(5):50–57, 2016.

Edward G. Coffman, Michael R. Garey, and David S. Johnson. Approximation algorithms for bin-packing — an updated survey. *Algorithm Design for Computer System Design*, pages 49–106, 1984.

Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017.

Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, 2019.

HKUST ADSL. kubernetes-scheduler-simulator: Simulate gpu-sharing scheduling policies for kubernetes clusters. <https://github.com/hkust-adsl/kubernetes-scheduler-simulator>, 2023.

IncidentHub. Major cloud outages of 2025. <https://blog.incidenthub.cloud/major-cloud-outages-2025>, 2025.

InfoQ. Aws disruption exposes fragility in critical cloud infrastructure. <https://www.infoq.com/news/2025/11/aws-disruption-cloud>, 2025.

Raj Jain, Dah-Ming Chiu, and William Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. *DEC Research Report TR-301*, 1984.

- David S. Johnson, Alan Demers, Jeffrey D. Ullman, Michael R. Garey, and Ronald L. Graham. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing*, 3(4):299–325, 1974.
- H. Lao, R. Xu, L. Chen, and J. Zhang. CAFGD: Reduce fragmentation in large-scale multi-tenant clusters for GPU sharing workloads. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 133–143, 2025.
- Francesco Lettich, Emanuele Carlini, Franco Maria Nardini, Raffaele Perego, and Salvatore Trani. Power- and fragmentation-aware online scheduling for GPU datacenters. In *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 43–52, 2025.
- Kanthi Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Mosharaf Chowdhury. Themis: Fair and efficient GPU cluster scheduling for ML training. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304, 2020.
- Ajay D. Mishra and Umesh Bellur. De-fragmenting the cloud. arXiv:1506.07020, 2015.
- Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498, 2020.
- Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*, 2018.
- Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021.
- Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC)*, 2012.
- Mohammad Shahradd, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218, 2020.
- X. Wang and Y. Wu. Survey on hypergraph algorithms for multi-resource allocation in cloud computing. *Journal of Computer Science and Technology*, 41(2), 2026.
- Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yan, and Liping Zhang. Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, 2023. URL <https://www.usenix.org/conference/atc23/presentation/weng>.
- Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong

Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.